7

Creating and Connecting a Knowledge Graph to an AI Agent

In the previous two chapters, we discussed the RAG framework in detail. We started with naïve RAG and then saw how we could add different components, replace others, or modify the entire pipeline for our needs. The whole system is extremely flexible, but some concepts remain the same. First, we start with a corpus (or multiple corpora of texts) and conduct embedding of these texts to obtain a database of vectors. Once the user query arrives, we conduct a similarity search on this database of vectors. Regardless of the scope or type of texts, our pipeline is based on the concept of vectorizing these texts in some way and then providing the information contained in the discovered texts to the LLM.

Texts are often full of redundant information, and in the previous chapter, we saw that LLMs are sensitive to the amount of noise in the input. Most people have seen the benefit of creating schematic notes or mind maps. These schematics are concise because of the principle that underlining everything in a book is like underlining nothing. The principle of these diagrams is to extract the key information to remember that will enable us to answer questions in the future. Schematics should present the fundamental information and the relationships that connect them. These schemas can be represented as a graph and, more precisely, as a knowledge graph. The advantage of these graphs is that they are compact, represent knowledge as entities and relationships, and we can conduct analyses and use graph search algorithms on them. Over the years, these **knowledge graphs** (**KGs**) have been built by major companies or institutions and are now available for use. Many of these KGs have been used for information extraction, where information is extracted with a series of queries to answer questions. This extracted information is a series of entities and relationships, rich in knowledge but less understandable to us humans. The natural step is to use this information for the context of an LLM and then generate a natural language response. This paradigm is called **GraphRAG,** and we will discuss it in detail in this chapter.

In any case, nothing prohibits us from using an LLM for all the steps in KG. In fact, LLMs have a number of innate capabilities that make them useful even for tasks for which they are not trained. This is precisely why we will see that we can use LLMs to extract relationships and entities and build our KGs. LLMs, though, also possess reasoning capabilities, and in this chapter, we will discuss how we can use these models to reason both about the information contained in graphs and about the structure of the graphs themselves. Finally, we will discuss what perspectives and questions remain open, and the advantages and disadvantages of the proposed approaches.

In this chapter, we'll be covering the following topics:

- Introduction to knowledge graphs
- Creating a knowledge graph with your LLM
- Retrieving information with a knowledge graph and an LLM
- Understanding graph reasoning
- Ongoing challenges in knowledge graphs and GraphRAG

Technical requirements

Most of this code can be run on a CPU, but it is preferable to be run on a GPU. The code is written in PyTorch and uses standard libraries for the most part (PyTorch, Hugging Face Transformers, LangChain, ChromaDB, **`sentence-transformer`**, **`faiss-cpu`**, and so on).

In this chapter, we also use Neo4j as the database for the graph. Although we will do all operations with Python, Neo4j must be installed and you must be registered to use it. The code can be found on GitHub: https://github.com/PacktPublishing/Modern-AI-Agents/tree/main/chr7.

Introduction to knowledge graphs

Knowledge representation is one of the open problems of AI and has very ancient roots (Leibniz believed that the whole knowledge could be represented and used to conduct calculations). The interest in knowledge representation is based on the fact that it represents the first step in conducting computer reasoning. Once this knowledge is organized in an orderly manner, it can be used to design inference algorithms and solve reasoning problems. Early studies focused on using deduction to solve problems about organized entities (e.g., through the use of ontologies). This has worked well for many toy problems, but it is laborious, often requires a whole set of hardcoded rules, and risks succumbing to combinatorial explosion. Because search in these spaces could be extremely computationally expensive, an attempt was made to define two concepts:

- **Limited rationality**: Finding a solution but also considering the cost of it
- **Heuristic search**: Limiting the search in space, thus finding a semi-optimal solution (a local but not global optima)

These principles have inspired a whole series of algorithms that have since allowed information searches to be conducted more efficiently and tractably. Interest in these algorithms grew strongly in the late 1990s with the advent of the World Wide Web and the need to conduct internet searches quickly and accurately. Regarding data, the World Wide Web is also based on three technological principles:

- **Distributed data**: Data is distributed across the world and accessible from all parts of the world
- **Connected data**: Data is interconnected and not isolated; the data's meaning is a function of its connection with other data
- **Semantic metadata**: In addition to the data itself, we have information about its relationships, and this metadata allows us to search efficiently

For this reason, we began to search for a technology that could respect the nature of this new data. It came naturally to turn to representations

of a graphical nature. In fact, by definition, graphs model relationships between different entities. This approach began to be used in web searches in 2012 when Google began adding knowledge cards for each concept searched. These knowledge cards can be seen as graphs of name entities in which the connections are the graph links. These cards then allow for more relevant search and user facilitation. Subsequently, the term *knowledge graph* came to mean any graph that connects entities through a series of meaningful relationships. These relationships generally represent semantic relationships between entities.

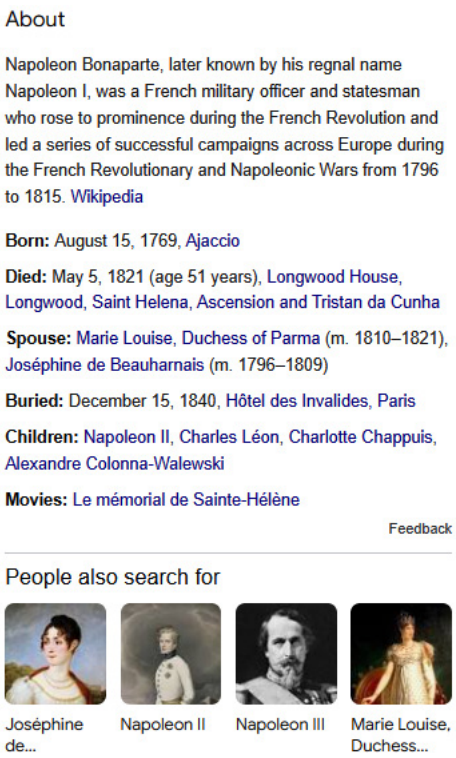


Figure 7.1 – Knowledge card in Google

A formal definition of graphs and knowledge graphs

Since KGs are a subtype of graphs, we will start with a brief introduction of graphs. Graphs are data structures composed of nodes (or vertices) that are connected by relationships (or edges) to represent a model of a domain. A graph can then represent knowledge in a compact manner while trying to reduce noise. There are different types of graphs:

- **Undirected**: Edges have no direction
- **Directed**: Edges have a defined direction (there is a defined beginning and end)
- **Weighted**: Edges carry weights, representing the strength or cost of the relationship
- **Labeled**: Nodes are associated with features and labels
- **Multigraph**: Multiple edges (relationships) exist between the same pair of nodes

The following figure shows a visual representation of these graphs:

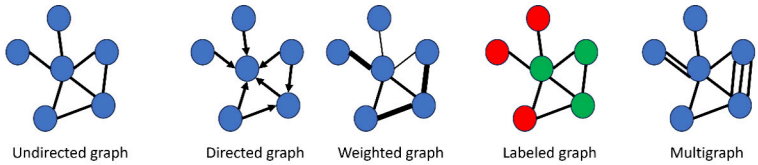
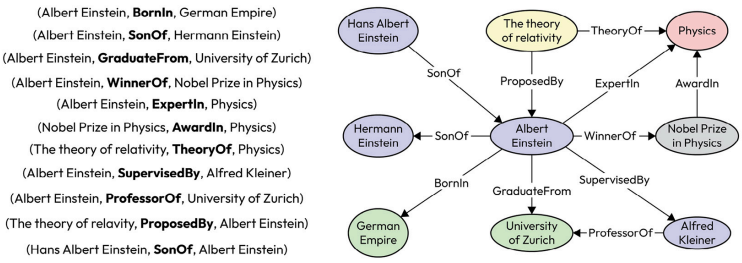


Figure 7.2 – Different types of graph architecture

A KG is thus a subgraph with three main properties:

- **Nodes represent real-world entities**: These entities can represent people, places, or domain-specific entities (genes, proteins, diseases, financial products, and so on)
- **Relationships define semantic connections between nodes**: For example, two people may be linked by a relationship that represents friendship, or a specific gene is associated with a particular disease
- **Nodes and edges may have associated properties**: For example, all people will have as a property that they are human beings (a label) but they can also have quantitative properties (date of birth, a specific identifier, and so on)

So, a little more formally, we can say that we have a knowledge base (a database of facts) represented in the form of factual triplets. A factual triples has the form of **`(head, relation, tail)`** or **`(subject, predicate, object)`**, or more succinctly, **`(e1,r1,e2)`**, such as **`(Napoleon, BornIn, Ajaccio)`**. The KG is a representation of this knowledge base that allows us to conduct interpretation. Given the structure of these triplets, a KG is a directed graph where the nodes are these entities and the edges are factual relationships.



(a) Factual triples in knowledge base. (b) Entities and relations in knowledge graph.

Figure 7.3 – Example of a knowledge base and knowledge graph (https://arxiv.org/pdf/2002.00388)

A KG is defined as a graph consisting of a set of entities, *E*, relations, *R*, and facts, *F*, where each fact *f* is a triplet:

$$\mathcal{G} = \{\mathcal{E}, \mathcal{R}, \mathcal{F}\}; f = (h, r, t)$$

As you can see, a KG is an alternative representation of knowledge. The same kind of data can be represented in either a table or a graph. We can directly create triplets from a table and then directly represent them in a KG. We do not need table headers, and it is easier to conduct the update of such a structure. Graphs are considered universal data representations because they can be applied to any type of data. In fact, not only can we map tabular data to a KG, but we can also get data from other formats (JSON, XML, CSV, and so on). Graphs also allow us to nimbly represent recursive structures (such as trees and documents) or cyclical structures (such as social networks). Also, if we do not have the information for all

properties, the table representation will be full of missing values; in a KG, we do not have this problem.

Graphs *per se* represent network structures. This is very useful for many business cases (e.g., in finance and medicine) where a lot of data is already structured as networks. Another advantage is that it is much easier to conduct a merge of graphs than of tables. Merging tables is usually a complicated task (where you have to choose which columns to merge, avoid creating duplicates, and other potential problems). If the data is in triplets, it is extremely easy to merge two KG databases. For example, look how simple it is to transform this table into a graph; they are equivalent:

Name	Married to
Jack	Charlotte
Charlotte	Jack

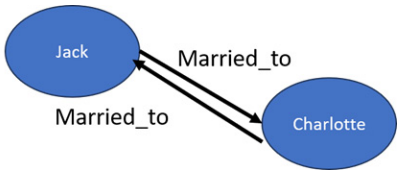


Figure 7.4 – Table and graph are equivalent

A KG should not be seen as a static entity. In fact, knowledge evolves; this causes new entities or relationships to be added. One of the most widely used tasks in **knowledge graph reasoning** (**KGR**) is to predict new relationships. For example, if A is the husband of B and father of C, this implies that B is the mother of C, which could be derived with logical ruling: *(A, husband of, B) ^ (A, father of C) -> (B, mother of, C)*. In this pre-existing datum, we have inferred a missing relationship.

Another very important task is how to conduct the update of a KG once we have obtained new triplets (this may require complex preprocessing). This is very important because once we have integrated some new knowledge, we can conduct further analysis and further reasoning. Also, being a graph, we can use graph analysis algorithms (such as centrality measures, connectivity, clustering, and so on) for our business cases. Leveraging these algorithms makes it much easier to conduct searches or complex queries in a KG than in relational databases.

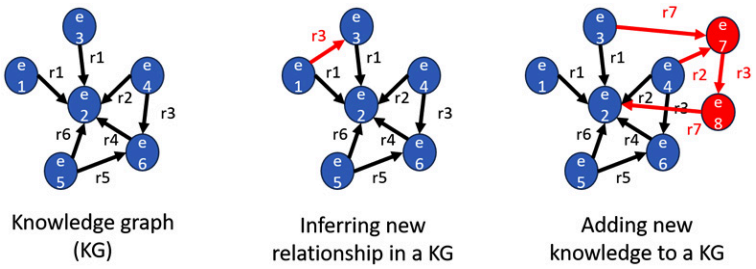


Figure 7.5 – A KG is a dynamic entity

In addition, KGs are much more flexible and adaptable than people think. KGs can be adapted for different tasks. For example, there are extensions of KGs such as **hierarchical KGs** where we have multiple levels. In hierarchical KGs, entities from one level can be connected to the next level (this, for example, is very useful when we have ontologies). Entities can also be **multimodal**, so a node can represent an image to which other entities (textual or other images, or other types of modalities) are connected. Another type of KG is a **temporal KG**, in which we incorporate a temporal dimension. This type of KG can be very useful for predictive analysis. We can see these KGs in the following figure:

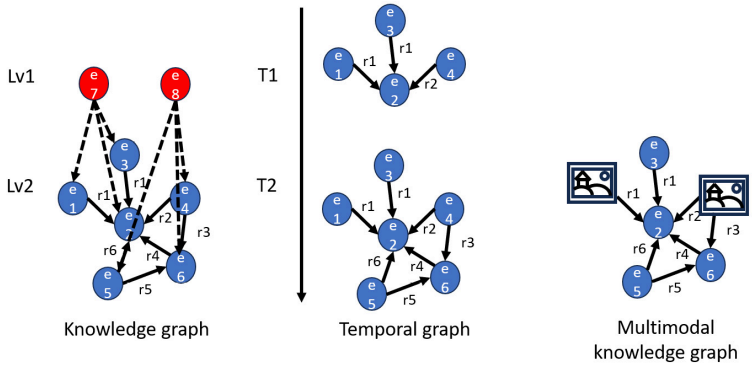


Figure 7.6 – Different types of knowledge graphs

Taxonomies and ontologies

The main difference between a graph and a KG is that the former is a given structure representing relationships between entities, while the latter makes semantic relationships explicit by allowing reasoning and inference to humans and machines. So, the advantage of a KG is that we can use algorithms for both graphs and specific reasoning algorithms (we will see later, in the *Understanding graph reasoning* section, some approaches in detail). These capabilities are enhanced by incorporating metadata. Indeed, we can construct **KG taxonomies**, which can be seen as hierarchical structures of similar semantic meaning (usually tree-like, for example, **`dog`** and **`cat`** entities might be grouped under **`mammals`**). In addition, multiple taxonomies can be integrated if necessary (thus having multiple trees to allow for more refined searching). These taxonomies help in searching or when we need to filter and work with very large KGs. **KG ontologies** are similar to taxonomies and may have a similar hierarchical structure, but they are a more flexible and expressive structure. In fact, an ontology is used to define the relationships, properties, and classification of a group of entities. It also allows rules to be defined about how these entities may interact (union, complement, cardinality, and transitive or intransitive properties of relationships). Ontologies also enable a shared vocabulary, thus allowing information to be integrated consistently. Ontologies can thus allow conducting reasoning and solving more complex problems. For example, we can add properties and then use these properties to solve a problem (Bob owns a car that has a **`maximum_speed`** property of **`100`** km, so Bob will not be able to get there in less than an hour because his job has a **`distance_from_home`** property of **`120`** km). Rules allow us to be able to improve search and solve tasks that were too complex before (for example, we can assign different properties to relations: if **`married_to`** is transitive, we can automatically infer information about a person without the relation being specified). Thanks to ontologies, we can conduct certain types of reasoning effectively and quickly, such as deductive reasoning, class inference, transitive reasoning, and so on.

Ontologies are generally grouped into two groups:

- **Domain-independent ontologies**: Ontologies that provide fundamental concepts that are not tied to a particular domain. They provide a high-level view that helps with data integration, especially when there are several domains. Commonly, these are a small number, they represent the first level, and they are the first ones built.
- **Domain ontologies**: These are focused on a domain and are used to provide the fundamental terminology. They are most useful for spe-

cialized domains such as medicine and finance. They are usually found at levels below the domain-independent one and are a subclass of it.

In this section, we have seen how KGs are flexible systems that can store data and be able to easily find knowledge. This flexibility makes them powerful tools for subsequent analysis, but at the same time, does not make them easy to build. In the next section, we will see how we can build a KG from a collection of texts.

Creating a knowledge graph with your LLM

The construction of a KG is generally a multistep process consisting of the following:

1. **Knowledge creation**: The first step, in which we define the purpose of this KG, is to gather the sources from which to extract knowledge. In this step, we have to decide how we build our KG but also where we maintain it. Once built, the KG has to be stored, and we have to have an efficient structure to query it.
2. **Knowledge assessment**: In this step, we assess the quality of the KG obtained.
3. **Knowledge cleaning**: There are several steps and procedures to make sure there are no errors and then correct them. This step can be conducted at the same time as knowledge assessment, and some pipelines conduct them together.
4. **Knowledge enrichment**: This involves a series of steps to identify whether there are gaps in knowledge. We can also integrate additional sources (extract information from other datasets, integrate databases, or merge multiple KGs).
5. **Knowledge deployment**: In this final step, the KG is deployed either as a standalone application (e.g., as a graph database) or used within another application.

We can see the process in the following figure:

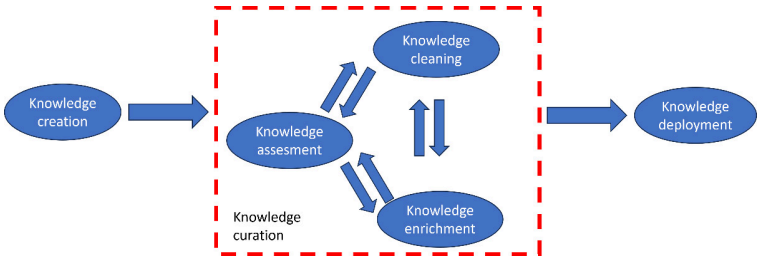


Figure 7.7 – KG construction pipeline

Knowledge creation

In general, when building a KG from scratch, the definition of ontologies is the first step. There are several guides on how to build them (both libraries and tools to visualize them). Efforts are made to build ontologies that are clear, verifiable, and reusable. Defining ontologies should be done with a purpose in mind, discussing what the purpose of a KG is with various stakeholders, and then defining ontologies accordingly. The most relevant ones should be chosen (the first level of the KG), following which

the hierarchy and properties should be defined. There are two approaches: top-down (define core ontologies first and then more specialized ones) or bottom-up (define specialized ontologies and then group them into superclasses). Especially for specialized domains, we could start from ontologies that have already been built (there are several defined for finance, medicine, and academic research) and this ensures better interoperability.

The next step is to extract knowledge from our sources. In this step, we have to extract triplets (or a set of facts) from a text corpus or another source (a database, or structured and unstructured data). Two tasks can be defined:

- **Named entity recognition** (**NER**): NER is the task of extracting entities from text and classifying them
- **Relation extraction** (**RE**): RE is the task of identifying connections between various entities in a context

NER is one of the most common tasks in **natural language processing** (**NLP**) and is used not only for KG creation but also as a key step when we want to move from unstructured text to structured data. It generally requires a pipeline consisting of several steps (text preprocessing, entity identification and classification, contextual analysis, and data post-processing). During NER, we try to identify entities by first conducting a preprocessing step to avoid errors in the pipeline (e.g., proper tokenization). Once entities are identified, they are usually classified (e.g., by adding a label such as people, organizations, or places). In addition, surrounding text is attempted to be used to disambiguate them (e.g., trying to recognize whether *Apple* in the text represents the fruit or the company). A preprocessing step is then conducted to resolve ambiguities or merge multi-token entities.

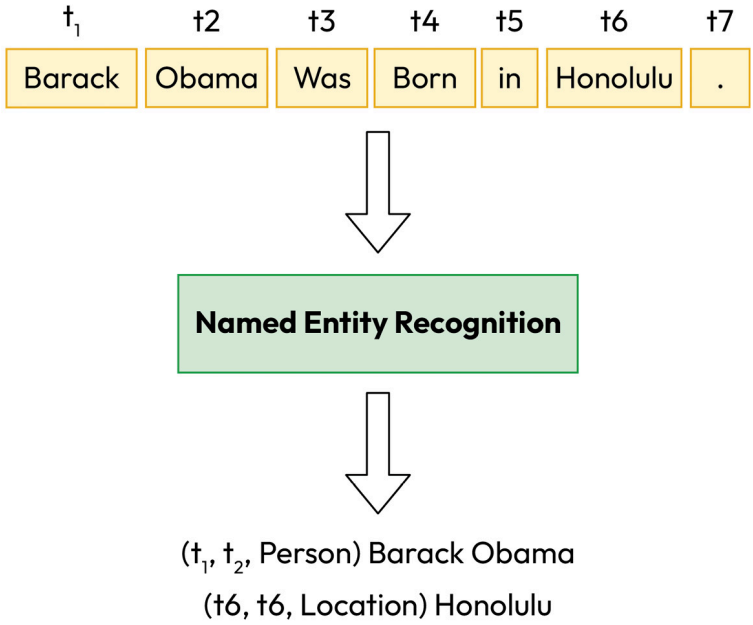


Figure 7.8 – Example of NER (https://arxiv.org/pdf/2401.10825)

RE is the task in which we understand the relationships between the various extracted entities. More formally, we use a model to identify and categorize the connections between entities in a text (e.g., in the sentence *Bob works at Apple*, we extract the relationship *works at*). It can be considered a separate task or, in some cases, conducted together with NER (e.g., with

a single model). Also, RE is a key step for KG creation and, at the same time, useful for several other NLP tasks (such as question answering, information retrieval, and so on).

There are several methods to be able to conduct NER and RE. The earliest and most laborious methods were knowledge-based or rule-based. For example, one of the simplest approaches to identifying company names in financial documents was to use indicators such as capital letters (identify `Mr.` and `Ms.` elements to extract people, and so on). Rule-based worked very well for standardized documents (such as clinical notes or official documents) but demonstrated little scalability. These methods require establishing laborious upstream rules and specific knowledge, risking missing many entities in different datasets.

Statistical methods based on the hidden Markov model, conditional random fields, or maximum entropy (methods that rely on predicting the entity based on likelihood learned from training data) have allowed for greater scalability. These methods, though, require large, quality datasets that have defined labels. Other supervised learning algorithms have been used to predict entities and then extract them. These algorithms have worked well with high computational costs and especially the need for labels. Obtaining labels is expensive and these datasets quickly become outdated (new companies, new products, and so on are created).

Recently, given the advances in unsupervised learning (models such as the transformer), it has been decided to use LLMs also for NER and RE and for constructing KGs (in some studies, these are called **LLM-augmented KGs**). The ability to process large corpora of text, the knowledge acquired during pre-training, and their versatility make LLMs useful for the construction of KGs (and other related tasks that we will see later).

Due to their ability to leverage contextual information and linguistic abilities, state-of-the-art methods generally employ transformer-based models for NER tasks. Previous methods had problems with texts that had complex structures (a token that belongs to several entities, or entities that are not contiguous in the text) while transformers are superior in solving these cases. BERT-based models were previously used, which were then later fine-tuned for different tasks. Today, however, we exploit the capabilities of an LLM that does not need to be fine-tuned but can learn a task without training through in-context learning. An LLM can then directly extract entities from text without the need for labels and provide them in the desired format (for example, we might want the LLM to provide a list of triplets or the triplet plus a given label). To avoid disambiguation problems, we can ask the LLM to provide additional information when conducting the extraction. For example, in music, *Apple* can refer to Apple Music, the British psychedelic rock band Apple, or the singer Fiona Apple. LLMs can help us disambiguate which of these entities it refers to based on the context of the period. At the same time, the flexibility of LLMs allows us to tie entities to various ontologies during extraction.

Similarly, an LLM can help with the RE task. There are several ways to do this. One of the simplest is to conduct sentence-level RE, in which you provide the model with a sentence and it must extract the relationship between the two entities. The extension of this approach is to extract all the relationships between entities at the level of an entire document. Since this is not an easy task, more sophisticated approaches with more than

one LLM can be used to make sure that we can understand relationships at the local and global levels of the document (for example, in a document, a local relationship between two entities is in the same sentence, but we can also have global relationships where an entity mentioned at the beginning of the document is related to an entity that is present at the end of the document).

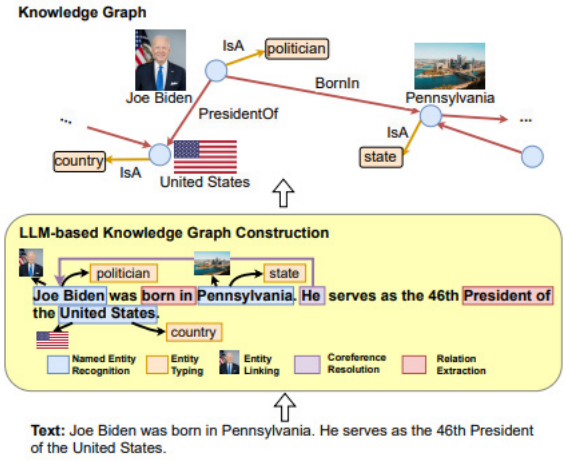


Figure 7.9 – General framework of LLM-based KG construction (This information was taken from an article published in 2023; https://arxiv.org/pdf/2306.08302)

As mentioned, the two tasks need not be conducted separately (NER and RE), but an LLM provides the flexibility to conduct them in a single step. In this case, it is of great importance to define the right prompt in which we instruct the model in extracting entities and relationships and in which format we want the output. We can then proceed iteratively, extracting entities and relationships for a large body of text. Alternatively, we can use a set of prompts for different tasks (one for entity extraction, one for relation extraction, and so on) and scan the corpus and these prompts automatically with the LLM. In some approaches, to maintain more flexibility, one LLM is used for extraction and then a smaller LLM is used for correction.

Another interesting perspective is that an LLM is enough to create a KG. In fact, LLMs are trained with a huge amount of text (the latest LLMs are trained with trillions of tokens that include scraping the internet and thousands of books). Several studies today show that even small LLMs (around 7 billion parameters) have considerable knowledge, especially about facts (the definition of knowledge in an LLM is also complicated because this is not associated with a single parameter but widespread). Therefore, some authors have proposed distilling knowledge directly from the LLM. In this case, by exploiting prompts constructed for the task, we conduct what is called a **knowledge search** of the LLM to extract triplets. In this way, by extracting facts directly from the LLM, we can then directly construct our KG. KGs constructed in this way are competitive in quality, diversity, and novelty with those constructed with large text datasets.

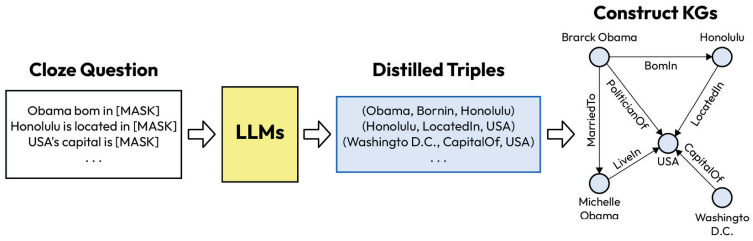


Figure 7.10 – General framework of distilling KGs from LLMs (https://arxiv.org/pdf/2306.08302)

Creating a knowledge graph with an LLM

In this tutorial, I will use Neo4j and LangChain to create a KG with an LLM. LangChain allows us to use LLMs efficiently to extract information from a text corpus, while Neo4j is a program for analyzing and visualizing graphs. The complete code is in the book's GitHub repository (https://github.com/PacktPublishing/Modern-AI-Agents/tree/main/chr7); here, we will describe the general process and the most important code snippets. We can have two methods:

- **Custom method**: LLMs have innate abilities to be able to accomplish tasks; we can take advantage of these generalist abilities
- **LangChain graph transformers**: Today, there are libraries that make the job easier and allow just a few lines of code to achieve the same result

The custom method is simply to define a prompt that allows the model to understand the task and execute it efficiently. In this case, our prompt is structured with the following elements:

- A clear definition of the task with a set of bullet points. The task description can contain both what the model must do and what it must not do.
- Additional context that allows the model to better understand how to perform the task. Since these models are trained for dialogic tasks, providing them with information about what role they should play helps the performance.
- Some examples to explain how to perform the task.

This approach builds on what we learned in ***Chapter 3***. The model we are using is instruction-tuned (trained to perform tasks) so providing clear instructions helps the model understand the task and perform it. The addition of some examples leverages in-context learning. Using a crafted prompt allows us to be flexible and be able to adapt the prompt to our needs:

```
#Custom method
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.messages import SystemMessage
from langchain_core.output_parsers import StrOutputParser
prompt = ChatPromptTemplate.from_messages([
    SystemMessage(content="""
    You are a helpful assistant in creates knowledge graphs by Generating Cypher Qu
    Task:
     *  Identify Entities, Relationships and Property Keys from Context.\n
     *  Generate Cypher Query to Create Knowledge Graph from the Entities Relations
     *  Extract ALL Entities and RelationShips as Possible.\n
     *  Always extract a person Profession as an Entity.\n
```

```
        *  Be creative.
        *  Understand hidden relationships from the network.
        Note: Read the Context twice and carefully before generating Cypher Query.\n
        Note: Do not return anything other than the Cypher Query.\n
        Note: Do not include any explanations or apologies in your responses.\n
        Note: Do not hallucinate.\n
        Entities include Person, Place, Product, WorkPlaces, Companies, City, Country,
        Few Shot Prompts:
         Example Context:
          Mary was born in 1995. She is Friends with Jane and John. Jane is 2 years ol
          Mary has a dog named Max,and is 3 years old. She is also married to John. Ma
         Answer:
           MERGE (Mary:Person {name: "Mary", birth_year: 1995})
           MERGE (Jane:Person {name: "Jane", age:1993})
           MERGE (John:Person {name: "John"})
           MERGE (Mary)-[:FRIENDS_WITH]->(Jane)
           MERGE (Mary)-[:FRIENDS_WITH]->(John)
           MERGE (Jane)-[:FRIENDS_WITH]->(Mary)
           MERGE (John)-[:FRIENDS_WITH]->(Mary)
           MERGE (Mary)-[:HAS_DOG]->(Max:Dog {name: "Max", age: 3})
           MERGE (Mary)-[:MARRIED_TO]->(John)
           MERGE (Mary)-[:HAS_PROFESSION]->(SoftwareEngineer:Profession {name: "Softwa
           MERGE (Mary)-[:FROM]->(USA:Country {name: "USA"})
      """),
      ("human", "Context:{text}"),
  ])
```

Executing the preceding should yield the following result:

```
MERGE (Emma:Person {name: "Emma Thompson"})
MERGE (NexCity:City {name: "NexCity"})
MERGE (Alex:Person {name: "Alex Martin", profession: "Entrepreneur"})
MERGE (Jessica:Person {name: "Jessica", profession: "Unknown"})
MERGE (Lily:Person {name: "Lily", profession: "Unknown"})
MERGE (Max:Person {name: "Max", profession: "Unknown", age: 5})
MERGE (Tom:Person {name: "Tom Wilson", profession: "Senior Developer"})
MERGE (Sarah:Person {name: "Sarah", profession: "Artist"})
MERGE (Maria:Person {name: "Maria Lopez", profession: "Research Institute Leader"})
MERGE (Jake:Person {name: "Jake Anderson", profession: "Chef"})
MERGE (Raj:Person {name: "Raj Patel", profession: "Data Scientist"})
MERGE (Nina:Person {name: "Nina", profession: "Unknown"})
MERGE (Leo:Person {name: "Leo Martinez", profession: "Financial Analyst"})
MERGE (Maya:Person {name: "Maya Chen", profession: "Startup Founder"})
MERGE (Helen:Person {name: "Helen Carter", profession: "Retired Teacher"})
MERGE (Chloe:Person {name: "Chloe Edwards", profession: "Unknown"})
MERGE (Liam:Person {name: "Liam", profession: "Journalist"})

MERGE (Emma)-[:FRIENDS_WITH]->(Alex)
MERGE (Alex)-[:PARTNER]->(Jessica)
MERGE (Alex)-[:CHILD]->(Lily)
MERGE (Alex)-[:CHILD]->(Max)
MERGE (Emma)-[:FRIENDS_WITH]->(Tom)
MERGE (Tom)-[:SPOUSE]->(Sarah)
MERGE (Emma)-[:FRIENDS_WITH]->(Maria)
MERGE (Emma)-[:FRIENDS_WITH]->(Raj)
MERGE (Raj)-[:PARTNER]->(Nina)
MERGE (Emma)-[:COLLABORATED_WITH]->(Maya)
MERGE (Emma)-[:NEIGHBOR]->(Helen)
MERGE (Chloe)-[:CHILD]->(Helen)
MERGE (Chloe)-[:MARRIED_TO]->(Liam)
```

Figure 7.11 – Screenshot of the results

The results show how a crafted prompt succeeds in generating triplets (which we can then use to construct our KG). This highlights the great flexibility of LLMs.

We do not always want a custom approach but might want to use a more established pipeline. LangChain provides the ability to do this with just a few lines of code:

```
from langchain_core.documents import Document
from langchain_experimental.graph_transformers import LLMGraphTransformer
llm_transformer = LLMGraphTransformer(llm=llm)
```

```
documents = [Document(page_content=content)]
graph_documents = llm_transformer.convert_to_graph_documents(documents)
```

LangChain gives us the same result in an already structured format that simplifies our work.

The graph can then be visualized in Neo4j and we can work on the graph, conduct searches, select nodes, and so on.

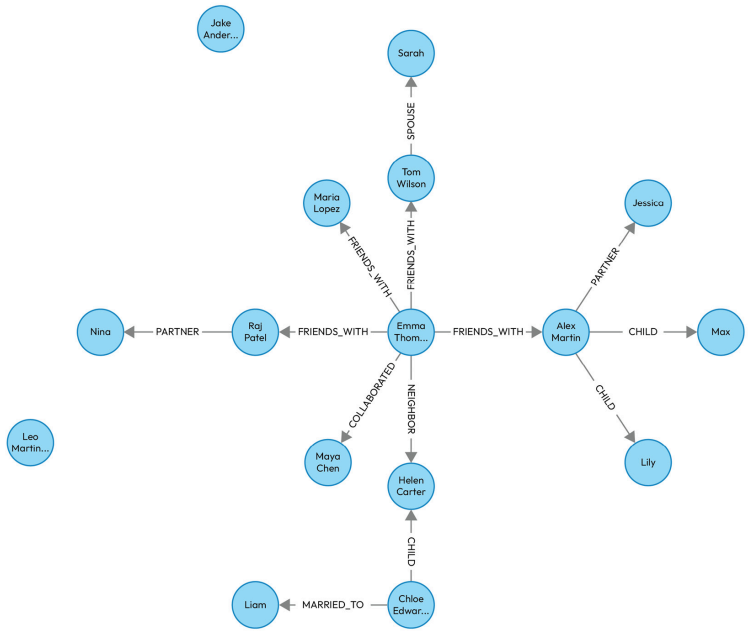


Figure 7.12 – Screenshot of the graph from Neo4j

Once the graph is generated, we can use it for our queries. Obviously, we can conduct these queries in Neo4j, but it is also possible to do it in Python. For example, LangChain allows us to conduct queries of our KG:

```
from langchain.chains import GraphCypherQAChain
graphchain = GraphCypherQAChain.from_llm(
    llm, graph=graph, verbose=True, return_intermediate_steps=True
)
results = graphchain.invoke({"query":"People who have kids"})
print(results["result"])
```

Once executed the code, you should obtain these results:

```
> Entering new GraphCypherQAChain chain...
Generated Cypher:
MATCH (p1:Person)-[:HAS_CHILD]->(p2:Person)
RETURN p1, p2
Full Context:
[{'p1': {'profession': 'Entrepreneur', 'name': 'Alex Martin'}, 'p2': {'profession': 'Unknown', 'name': 'Lily
'}}, {'p1': {'profession': 'Entrepreneur', 'name': 'Alex Martin'}, 'p2': {'profession': 'Unknown', 'name': '
Max', 'age': 5}}]

> Finished chain.
Alex Martin, Max have kids.
```

Figure 7.13 – Querying the KG

As we can see, LangChain in this case generates a corresponding query in Cypher and then conducts the graph query. In this way, we are using an LLM to generate a query in Cypher, while we can write directly in natural language.

Knowledge assessment

Once the KG has been created, it is necessary to check for errors and the overall quality of the KG. The quality of a KG is assessed by a set of dimensions (there are metrics associated with each of these dimensions) that are used to monitor the KG in terms of accessibility, representation, context, and intrinsic quality. Some of the metrics are as follows:

- **Accuracy**: This metric assesses accuracy in syntactic and semantic terms.
- **Completeness**: This metric measures how much knowledge a KG contains with respect to a certain domain or task. It usually measures whether a KG contains all the necessary entities and relationships for a domain (sometimes a comparison with a golden standard KG is used).
- **Conciseness**: KGs allow knowledge to be expressed efficiently, but they risk scaling quickly. Blank nodes (specific types of nodes that represent anonymous or unnamed entities, used when a node is needed in the graph but is not precisely indicated) can often be generated during the creation process. If care is not taken, one risks filling the KG with blank nodes.
- **Timeliness**: Knowledge should also be updated regularly because it can change and become outdated. Therefore, it is important to decide the frequency of updates.
- **Accessibility, ease of manipulation, and operation**: KGs are used for searches or other tasks; metrics exist today that measure the usefulness of KGs. In fact, for a KG to be useful, it must be easily accessible, be able to be manipulated, and be able to conduct research and updates.
- **Ease of understanding**: Since the KG is meant to be used to represent knowledge for humans, some authors have proposed measuring the degree to which the KG is interpretable for humans. Indeed, today, there is a greater emphasis on transparency and interpretation of models in AI.
- **Security, privacy, and traceability**: Metrics also exist today to control who accesses the KG and whether it is secure from outside access. Similarly, knowledge also needs to be tracked, as we need to be sure which sources it comes from. Traceability also allows us to be in privacy compliance. For example, our KG may contain sensitive data about users or come from erroneous or problematic documents. Traceability allows us to correct these errors, delete data from users who require their data to be deleted, and so on.

Knowledge cleaning

Having assessed the quality of our KG, we can see that there are errors. In general, error detection and correction are together called **knowledge cleaning**. Different types of errors can occur in a KG:

- We may have entities or relationships that have syntactic errors
- Some errors may be ontology-related (assigning to ontologies that do not exist, connecting them to the wrong ontologies, wrong properties of ontologies, and so on)
- Some may be semantic and may be more difficult to identify
- There are also knowledge errors that may result from errors in the sources for creating the knowledge (symptom *x* is not a symptom of disease *y*, person *x* is not the CEO of company *y*, and so on)

There are several methods to detect these errors. The simplest methods are statistical and seek to identify outliers in a KG by exploiting probabilistic and statistical modeling. There are also more sophisticated variations that exploit simple machine-learning models. These models are not particularly accurate. Since we can use logical reasoning and ontologies with KGs, there are knowledge-based reasoning methods to identify outliers (e.g., an instance of a person cannot also be an instance of a place, so by exploiting similar rules, we can identify outliers). Finally, there are methods based on AI, and one can also use an LLM to check for errors. An LLM possesses both knowledge and reasoning skills so it can be used to verify that facts are correct. For example, if for some error, we have the triplet **`(Vienna, CapitalOf, Hungary)`**, an LLM can identify the error). Then, there are similar methods for conducting KG correction. However, several frameworks have already been built and established to conduct detection and correction.

Knowledge enrichment

Knowledge enrichment (or KG completion) is generally the next step. KGs are notoriously incomplete, so several rounds of KG completion and correction can occur. The completeness of a KG is sometimes complicated to define and is contextual to the domain and application tasks. The first step in completing a KG is usually to identify additional sources of information (e.g., for a medical KG, it could be an additional biomedical database or an additional corpus of scientific articles). Often, in the first step of building the KG, we use only one data type (unstructured text) and then extend the extraction in a second step to other data types (CSV, XML, JSON, images, PDF, and so on). Each of these data types presents different challenges, so we should modify our pipeline. The more sources we use, the more crucial KG cleaning and alignment tasks become. For example, the more heterogeneous the sources, the greater the importance of entity resolution (identifying duplicate entities at the KG level).

An interesting alternative is to infer knowledge using an LLM (or other transformer models). For example, three possible approaches have been explored:

- **Joint encoding**: The triplet **`(h,r,t)`** is given to a model as a transformer model to predict the probability that it exists (**`0`** represents that the triplet is invalid, while **`1`** is a valid triplet). A variation of this approach is to take the final hidden state from the model and train a linear classifier to predict in a binary fashion whether the triplet is valid or not.
- **Masked language model (MLM) encoding**: Instead of conducting the entire embedding, one of the components of the triplet is masked and the model must predict it. In other words, if we have **`(h,r,?)`**, we can try to complete the gap.
- **Separated encoding**: The triplet is separated into two components, **`(h,r)`** and **`(t)`**. In this way, we get two representations from the model (the final hidden state of the model is used). After that, we use a scoring function to predict whether this triplet is valid. This approach is definitely more accurate but risks a combinatorial explosion. As you can see, in this approach, we are trying to calculate the similarity between two textual representations (the representation of **`(h,r)`** and **`(t)`**).

These approaches are, in fact, very similar to those we have seen in previous chapters when trying to calculate the similarity between two sentences.

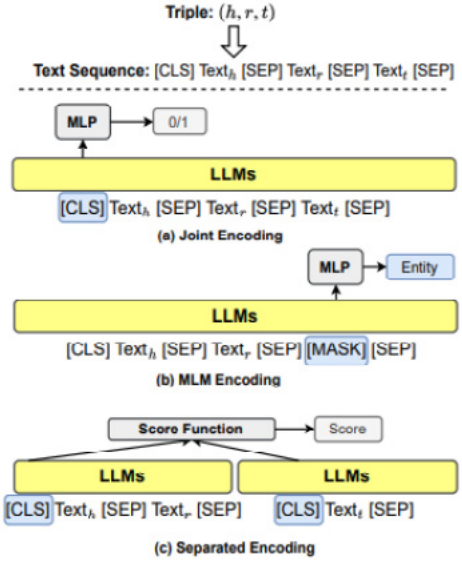


Figure 7.14 – LLMs as encoders for KG completion (https://arxiv.org/pdf/2306.08302)

Alternatively, one can use few-shot examples or other prompting techniques and ask an LLM to complete them directly. In addition, this approach allows you to be able to provide additional items in the prompt. In previous approaches, we provided only the triplet `(h,r,t)`; with prompt engineering, we can also provide other contextual elements (relationship descriptions, entity descriptions, etc.) or add instructions to better complete the task.

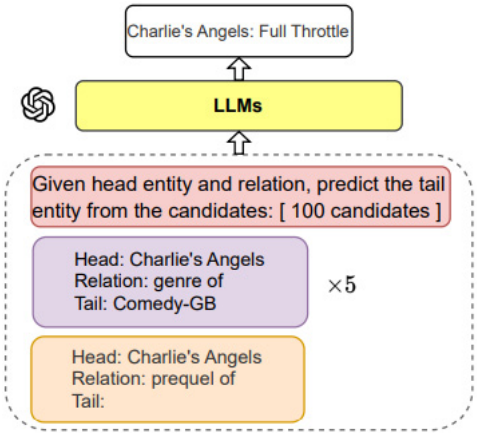


Figure 7.15 – Prompt-based KG completion (https://arxiv.org/pdf/2306.08302)

Knowledge hosting and deployment

The last step is the hosting and deployment of the KG. The KG is a set of nodes and relationships, so we can use a graph-specific paradigm to be able to store the data. Of course, hosting a KG is not without challenges:

- **Size**: The larger the KG, the more complex its management becomes
- **Data model**: We have to choose the system that allows us to optimally access the information for our tasks, as different systems have different advantages and disadvantages

- **Heterogeneity**: The graph may contain multiple modes, thus making storage more complex
- **Speed**: The more it grows, the more complex knowledge updates become
- **User needs**: Users may have heterogeneous needs that may be conflicting, requiring us to have to implement rules and constraints
- **Deployment**: The system must be accessible to users and allow easy inference

There are different alternatives for the storage of a KG:

- KGs can be hosted in classic **relational databases** (e.g., **Structured Query Language** (**SQL**) where entities and relationships are stored in tables). From these tables, one can then reconstruct the graph's relational structure using projections. Using a relational database to store a large KG can result in large tables that are impractical or a multitude of tables with a complex hierarchy.
- An alternative is the **document model** where data is stored as tuples (key-value pairs) and then organized into collections. This structure can be convenient for searching; it is a schematic system that allows the speed of writing, but updating knowledge in nested collections can be a nightmare.
- **Graph databases** are databases that are optimized for storing and searching graphs and data transformation. The graph data model then has nodes and edges with various metadata that are attached. The query language is also adapted to this structure (and is vaguely reminiscent of SQL). Graph databases also have the advantage of allowing heterogeneity and supporting speed. Neo4j is one of the most widely used and uses an adapted query language (Cypher).
- **Triplet stores** are where the database is made up directly of triplets. Databases exist today that save information in triplets and allow queries to be conducted in the database. Typically, these databases also have native support for ontologies and for conducting logical reasoning.

Hosting comes with its own set of challenges, and the choice of data model should be conducted with subsequent applications in mind. For example, if our KG needs to retrieve data from our relational database, using this system has advantages for integration. On the other hand, though, in this case, we will sacrifice performance for heterogeneity and speed. A graph database handles performance and the structural nature of the graph better, but it may integrate poorly with other components of the system. Whatever system we use for storage, we can either build hybrid systems depending on the applications or create a KG as a layer on top of the database.

Deployment is the last step in the pipeline. That doesn't mean it's the end of the story, though. A KG can become outdated easily, so we need to have pipelines in mind for knowledge updates or to be able to handle new applications. Similarly, the entry of new knowledge means that we must have pipelines for knowledge assessment (monitoring the quality of the KG, ensuring that no errors are entered or that there are no conflicts). Some knowledge may be outdated or need to be deleted for legal or privacy issues; therefore, we should have pipelines for cleaning the KG. Other pipelines should instead focus on controlling access and security of our system.

In this section, we have seen all the steps necessary to create and deploy a KG. Now that we have our KG, we can use it; in the next section, we will discuss how to find information and use it as a context for our LLM.

Retrieving information with a knowledge graph and an LLM

In the previous two chapters, we discussed the capabilities of RAG and its role in reducing hallucinations generated by LLMs. Although RAG has been widely used in both research and industrial applications, there are still limitations:

- **Neglecting relationships**: The text in the databases is interconnected and not isolated. For example, a document is divided into chunks; since these chunks belong to a single document, there is a semantic connection between them. RAG fails to capture structured relational knowledge when this cannot be captured by semantic similarity. Some authors point out that, in science, there are important relationships between an article and previous works, and these relationships are usually highlighted with a citation network. Using RAG, we can find articles that are similar to the query but we cannot find this citation network, losing this relational information.
- **Redundant information**: The context that comes to the LLM is a series of concatenated chunks. Today with LLMs, we can add more and more context (the context length of models is getting longer and longer) but they struggle with the presence of redundant information. The more chunks we add to the context, the greater the amount of redundant or non-essential information to answer the query. The presence of this redundant information reduces the performance of the model.
- **Lacking global information**: RAG finds a set of documents but fails to find global information because the set of documents is not representative of global information. This is a problem, especially for summarization tasks.

Graph retrieval-augmented generation (**GraphRAG**) has emerged as a new paradigm to try to solve these challenges. In traditional RAG, we find text chunks by conducting a similarity analysis on the embedded vectors. In GraphRAG, we conduct the search on the knowledge graph, and the found triplets are provided to the context. So, the main difference is that upon arrival of a query from a user, we conduct the search in the KG and use the information contained in the graph to answer the query.

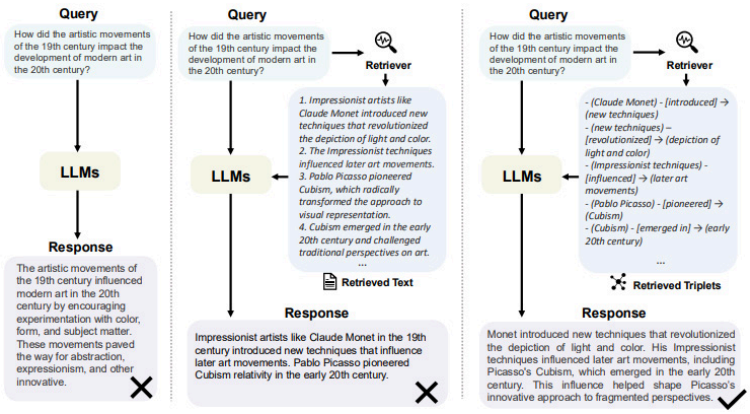


Figure 7.16 – Comparison between direct LLM, RAG, and GraphRAG (https://arxiv.org/pdf/2408.08921)

Formally, we can define GraphRAG as a framework that exploits a KG to provide context to an LLM and produce a better response. The system, therefore, is very similar to classical RAG; to avoid confusion, in this context, we will call it *vector RAG*. In GraphRAG, the KG is the knowledge base, and from this, we find information on entities and relationships. GraphRAG consists of three main steps:

1. **Graph-based indexing** (**G-indexing**): In this initial phase, the goal is to build a graph database and index it correctly.
2. **Graph-guided retrieval** (**G-retrieval**): Once we have a KG, when a query arrives, we have to find and extract the various information needed to answer the query. Given a query, **q**, in natural language, we want to extract a subgraph that we can use to correctly answer the query.
3. **Graph-enhanced generation** (**G-generation**): The last step concerns using the found knowledge for generation. This step is conducted with an LLM that receives the context and generates the answer.

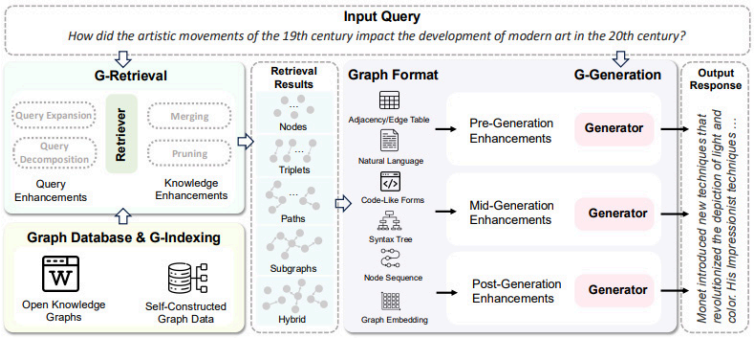


Figure 7.17 – Overview of the GraphRAG framework for a question-answering task (https://arxiv.org/pdf/2408.08921)

In the following subsections, we will talk about each step in detail.

Graph-based indexing

In the first step, we need to choose what our graph data will be. Generally, two types of KGs are used: open KGs or self-constructed KGs. In the first case, we can use a KG that is already available and adapt it to our GraphRAG. Today, many KGs have already been built and are available (for example, Wikidata is a knowledge base that collects data from various Wikipedia-related projects). Several KGs are specialized in a particular domain; these KGs have a greater understanding of a particular domain (some of these KGs are open and usable). Alternatively, it is possible to build your own KG.

When building it or before using it in GraphRAG, you should pay attention to indexing. Proper indexing allows us to have a faster and more efficient GraphRAG. Although we can imagine the KG visually as a graph, it is still stored in a database. Indexing allows us access to the information we want to find. Thus, there are several types of indexing. For example, we can have text descriptions associated with nodes, triplets, or ontologies that are then used during the search. Another way is to transform graph data into vectors and conduct the search on these vector spaces (embed-

ding). We can also use indexing that better respects the graph nature of the data or hybrid versions.

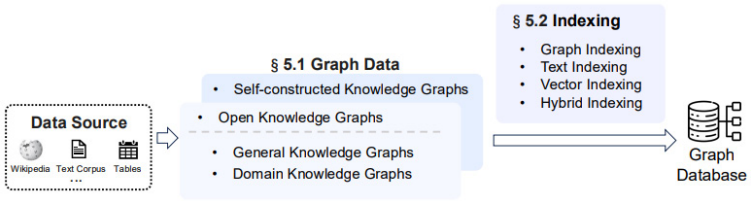


Figure 7.18 – Overview of graph-based indexing (https://arxiv.org/pdf/2408.08921)

Graph-guided retrieval

In GraphRAG, retrieval is crucial for the quality of response generation (similar to vector RAG). The search for a KG has two challenges that need to be solved:

- **Explosive candidate subgraphs**: As the graph grows, the number of subgraphs in the KG increases exponentially. This means we need efficient algorithms to explore the KG and find the relevant subgraphs. Some of these algorithms use heuristic methods to be more efficient.
- **Insufficient similarity measurement**: Our query is in text form, but we want to conduct our similarity search on a graph. This means that our algorithm must be able to understand both textual and structural information and be able to succeed in comparing the similarity between data from different sources.

We can have different types of retrievers. The simplest are **non-parametric retrievers**. These are based on the use of heuristic rules or other traditional graph search algorithms. For example, given a query, we search for entities present and take *k*-hop paths (in simple words, for the query *Where is Obama born?*, starting from the **`Obama`** entity, in KG, we take neighboring entities, **`k-hop=1`**, or even neighbors of neighbors, **`k-hop=2`**, and so on). Non-parametric retrievers are the simplest and also the fastest systems, but they suffer from inaccurate retrieval (they can be improved by learning). There are machine and deep learning models that are natively trained on graphs. GNN-based retrievers are one example. **Graph neural networks (GNNs)** are neural networks that natively handle graphs and can be used for many tasks on graphs (node classification, edge prediction, and so on) so they search the graph for subgraphs similar to the query.

Alternatively, we can use an LLM-based retriever where we have a transformer-based model that conducts the search. The model then processes and interprets the query to conduct the search. Several of these LLMs are models that have been trained on the text, and then fine-tuning is conducted to search the graphs. One advantage is that an LLM can be used as an agent and use different tools or functions to search the graph. Both LLM-based retrievers and GNN-based retrievers significantly improve retrieval accuracy but at a high computational cost. There are also alternatives today that use different methods (conduct with both a GNN and LLM, or use heuristic methods together with an LLM) or the process can be multistage (e.g., conduct an initial search with an LLM and then refine the results).

As was done for the vector RAG, we can add additional components to conduct enhancement. For example, in the previous chapter, we saw that we can rewrite a query or decompose queries that are too complex. Query modification helps to better capture the meaning of the query (because sometimes the query does not capture the implicit meaning intended by the user). Retrieval can also be a flexible process. In the previous chapter, we saw that in naïve RAG, retrieval was conducted only once, but then variations in advanced and modular RAG were established where retrieval can be multistage or iterative. Even more sophisticated variations make the process adaptive depending on the query, so for simpler queries, only one retrieval is conducted, and for more complex queries, multiple iterations may be conducted. Similarly, the results obtained after retrieval can also be modified. For example, even with GraphRAG, we can conduct a compression of the retrieved knowledge. In fact, we may also find redundant information if we conduct multiple retrieval stages and thus it is convenient to filter out irrelevant information.

Today, there are also reranking approaches to reorder the retrieved results with GraphRAG. One example is to reorder the various subgraphs found and perhaps choose the top *k* subgraphs.

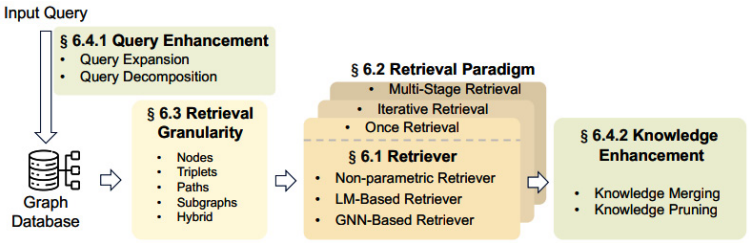


Figure 7.19 – General architectures of graph-based retrieval (https://arxiv.org/pdf/2408.08921)

Another difference with vector RAG is how we control search granularity. In vector RAG, granularity is controlled by deciding the size of the chunks. In the case of GraphRAG, we do not conduct chunking or find chunks. We can, however, control granularity during retrieval by choosing what we find:

- **Nodes**: In GraphRAG, we can retrieve individual entities. Nodes can have properties associated with them and then only add entities and their properties to the context. This can be useful for target queries.
- **Triplets**: By expanding the search granularity, we choose to retrieve triplets (so not only nodes, but also their relationships). This is useful when we are interested not only in the entity itself but also in their relationships.
- **Paths**: In this case, we still expand the retrieval. A path is a chain of nodes and relationships, so starting from entity *X* and arriving at entity *Y*, the path is all the chain of entities and relationships that connect them. Obviously, there are multiple paths between different entities, and these grow exponentially as the size of the graph increases. So, we generally define rules, use GNNs, or choose the shortest path.
- **Subgraphs**: A subgraph can be defined as a subset of nodes and relationships internal to the KG. Extracting a subgraph allows us to answer complex queries because it allows us to analyze complex patterns and dependencies between entities. There are several ways to extract a subgraph: we can use specific patterns or conduct a merge of different paths.

- **Hybrid granularities**: We can use different granularities at the same time or choose an adaptive system.

In the case of GraphRAG, balancing granularity and efficiency is important. We do not want to saturate the context with elements to prevent later LLM struggles with irrelevant information. It also depends on the complexity of the query: for simple queries, even low granularity is enough, whereas complex queries benefit from higher granularity. An adaptive approach can make the system more efficient while maintaining nuances when needed.

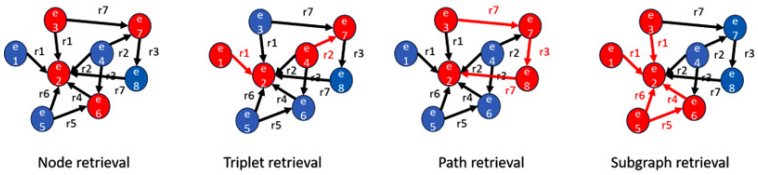


Figure 7.20 – Different levels of retrieval granularity

Once the knowledge is found and cleaned up, we can provide it to the LLM to generate a response to the query. This knowledge enters the prompt provided to the LLM and the model generates a response. Alternatively, this found knowledge can be used for certain types of models that are used for some specific tasks (e.g., a GNN to answer multiple-choice questions). The main problem is that the graph has a non-Euclidean nature and integration with textual information is not optimal. For this, graph translators that convert the found graph information into more digestible information for an LLM can be used. This conversion increases the LLM's ability to understand the information. So, once we find the information in graph form (nodes, relationships, path, or subgraph), we put it in the context of the LLM. There are some alternatives:

- **Graph formats**: We can directly add the set of relationships and nodes in the prompt, or we can use a form of graph structure representation such as adjacency or edge tables. The latter compactly conveys a better relational structure. Another idea is a node sequence, which is generated according to a predeterminate rule. This is a compact representation that contains the order of the nodes in the graph.
- **Natural language**: There are specific graph languages that can be used to transform information into natural language, a representation that is more congenial to LLM. In this process, we convert the found subgraph into a descriptive form. Templates can be used to transform the graph where it is filled with nodes and relationships. In some templates, you can define which are the nearest neighbors of a node and which are the most distant (1-hop and 2-hop in the graph), or you can use an LLM to transform this subgraph into a natural language description.
- **Syntax tree**: The graph is flattened and represented as a syntax tree. These trees have the advantage of a hierarchical structure and maintain the topological order of the graph. This approach maintains the properties of the graph but makes the information more digestible for the LLM.
- **Code-like forms**: Retrieved graphs can be converted into a standard format such as **Graph Markup Language (GraphML)**. These languages are specifically designed for graphs but are a structural and textual hybrid.

Conversion still presents difficulties because it must ensure that the result is concise and complete but, at the same time, understandable by the LLM. Optimally, this representation should also include the structural information of the graph. Whether the retrieved subgraph is converted or not, the result is entered into the LLM prompt and a response is generated.

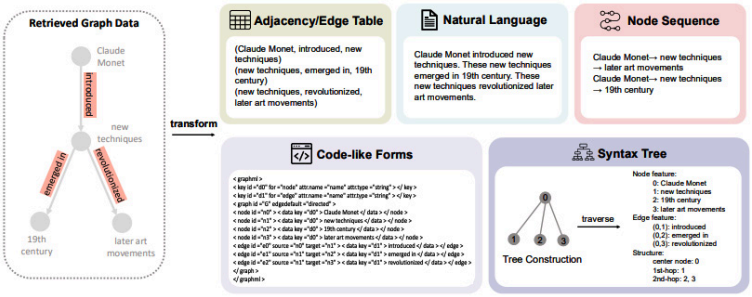


Figure 7.21 – Subgraph transformation to enhance generation (https://arxiv.org/pdf/2408.08921)

GraphRAG applications

GraphRAG has several applications. The first is question answering (so the same application as RAG) where we extract the subgraphs and the LLM uses them for subsequent reasoning and answering. A sub-branch of question answering is commonsense reasoning question answering where it often takes the format of multiple choice questions. For this subtask, we often do not use an LLM but a GNN or other machine learning (ML) model instead. However, KGs (and therefore also GraphRAG) have an extensive application for information retrieval, for example, if we want to investigate the relationships between some entities of interest. A KG can be used by itself to extract relationships between entities, but the addition of generation with an LLM allows us to explore these relationships and contextual nuances better. This is an attractive factor for academic and literature research. In fact, in academia, an article is authored by multiple authors who are part of different institutions. An article builds on previous research, and so for each article, there is a network of citations. These structural elements are easily modeled for a graph. An interesting application recently published shows how Ghafarollahi et al. used multiple KG agents to analyze published literature and propose new research hypotheses. In short, they extracted paths or subgraphs from a KG (constructed from 1,000 articles), then an agent analyzed the ontologies, and then a new research hypothesis was generated from this. In this interesting application, a number of agents collaborate to create new potential searches for new materials.

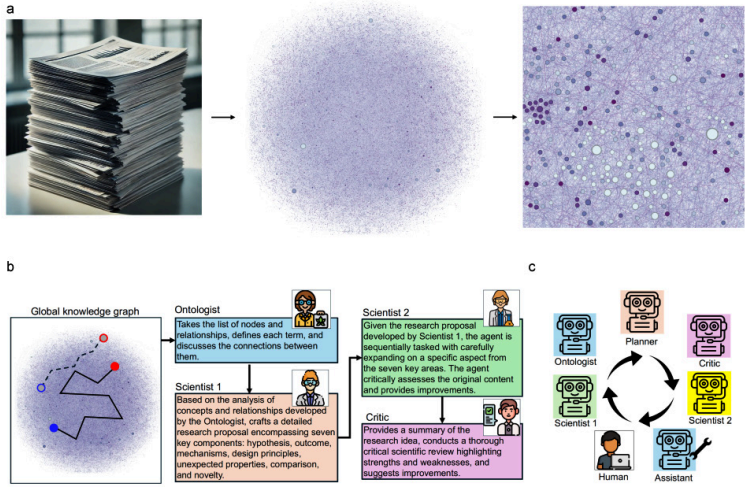


Figure 7.22 – Overview of the multi-agent graph-reasoning system for scientific discovery assistance (https://arxiv.org/pdf/2409.05556v1)

One of the reasons for interest in GraphRAG is that KGs used to be used for fact verification (after all, a KG is a collection of facts) so providing facts to an LLM should reduce LLM hallucinations. This aspect makes it particularly attractive for biomedical applications. In fact, hallucinations are a serious problem for medical decision-making applications. In medicine, if the vector RAG can reduce hallucinations, it does not allow for a holistic view, especially when an overview is needed to answer a question. Therefore, Wu et al. suggest using a GraphRAG-based approach called **MedGraphRAG**. In this work, they use several medical sources to create their system and take advantage of the hierarchical nature of KGs. They construct three levels for their KG. At the first level, there are user-provided documents (medical reports from a hospital). Entities at this level are then connected to a more foundational level of commonly accepted information. The second level is constructed from medical textbooks and scientific articles. Finally, at the third level, there are well-defined medical terms and knowledge relationships that have been obtained from standardized and reliable sources. Leveraging retrieval from this KG obtains state-of-the-art results on major medical question-answering benchmark datasets. The advantage is that this system also outperforms models that are fine-tuned on medical knowledge, thus with large computational savings.

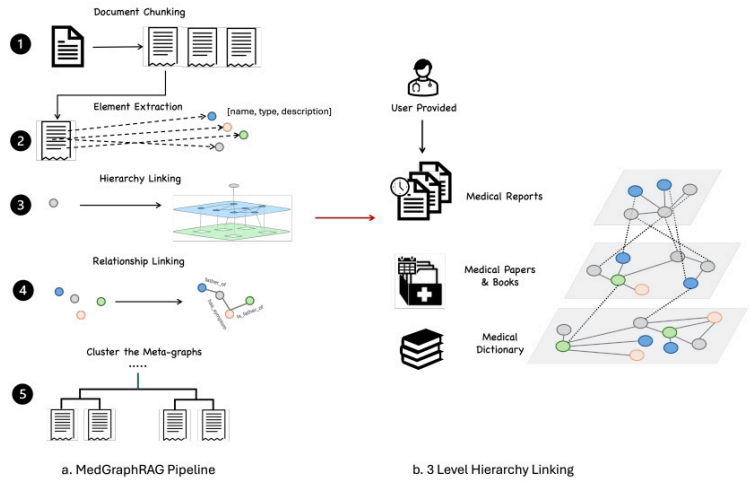


Figure 7.23 – MedGraphRAG framework (https://arxiv.org/pdf/2408.04187)

A further interest in GraphRAG comes from the use of KGs to propose recommendations to users. In e-commerce platforms, recommendation sys-

tems are used to predict the future purchasing intentions of users and suggest other products of interest. Thus, it was proposed that the system matches a new user to subgraphs derived from past users with similar behavior, and leverages these to predict likely future purchases and suggest appropriate products. In addition, this approach can also be useful for the legal and financial fields. In the legal field, there are extensive citations between cases and judicial opinions, and judges use past cases and opinions to make new decisions. Given a legal case, GraphRAG could suggest previous legal cases and help in decision-making. In finance, GraphRAG might suggest previous financial transactions or customer cases.

Finally, up to this point, we have suggested that GraphRAG and vector RAG are antagonistic. Actually, both systems have advantages and disadvantages so it would be more useful to use them in synergy. Sarmah et al., proposed **HybridRAG**, where both GraphRAG and vector RAG are used in one system. Their system shows advantages in financial responses. In the future, there may be systems that exploit both one and the other approach with the addition of a router that can choose whether to search for the KG or the vector database. Alternatively, there could be more complex systems for knowledge fusion in context (especially if KG search and chunks provide some redundant information).

In this section, we discussed how an LLM can be connected to a KG, and how it can be used to find information that enriches the context of the LLM. In the next section, we will discuss other tasks for which the synergy of LLMs and KGs is useful.

Understanding graph reasoning

This section is devoted to a discussion of how to solve graph data tasks. In this section, we will discuss some of the approaches used to solve tasks on knowledge graphs: KG embeddings, GNNs, and LLMs. KG embeddings and GNNs would require at least one chapter each; hence, these topics are outside the scope of the book, but we believe that an introduction to them would be beneficial to a practitioner. In fact, both embedding and GNNs can be used synergistically with LLMs and agents.

There are many tasks in which a model is required to understand the structure to solve, and these are collectively called **graph structure understanding tasks**. Many of these tasks are solved using algorithms or models designed specifically to learn these tasks. Today, a new paradigm is being developed in which we try to use LLMs to solve these tasks; we will discuss this in depth at the end of this section. Examples of tasks might be degree calculation (how many neighbors a node has), path search (defining a path between two nodes, calculating which is the minimum path, and so on), Hamilton path (identifying a path that visits each node only once), topological sorting (identifying whether nodes can be visited in topological order), and many others. Some of these tasks are simple (degree calculation and path search) but others are much more complex (topological sorting and Hamilton path).

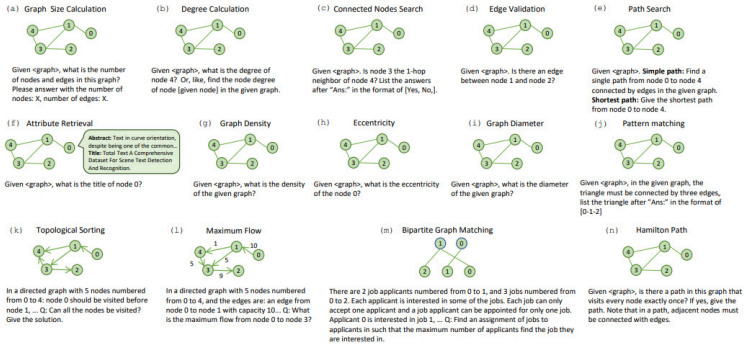


Figure 7.24 – Graph structure understanding tasks (https://arxiv.org/pdf/2404.14809)

Graph learning tasks, on the other hand, require the model to include not only the structure of the graph but also the attributes of the graph (features of nodes, edges, and the graph), thus understanding the semantic information of graphs. Examples of some of the tasks are node classification (classify the node according to its attributes and according to the attributes of its neighbors), graph classification (you have to understand the whole graph to classify it), edge classification, and node feature explanation (explain a feature of the node). **Knowledge graph question answering (KGQA)** is a task that falls into this group, as we need to understand both the structure and meaning of entities and relationships to answer questions. A similar task is conducting KG queries to generate text (this can also be seen as a subtask). KG embeddings capture multi-relational semantics and latent patterns in the graph, making them particularly useful for relational reasoning and symbolic link prediction tasks (KG link prediction, for example). GNNs, on the other hand, capture graph structure and node/edge features; these make them perform well for tasks that require inductive reasoning, use of features, or local/global representation of graph structure (node or graph classification/regression).

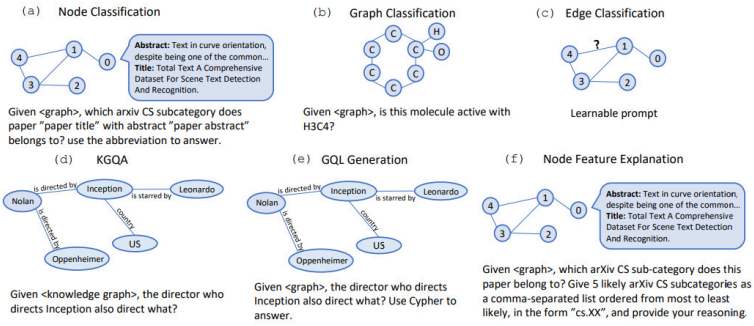


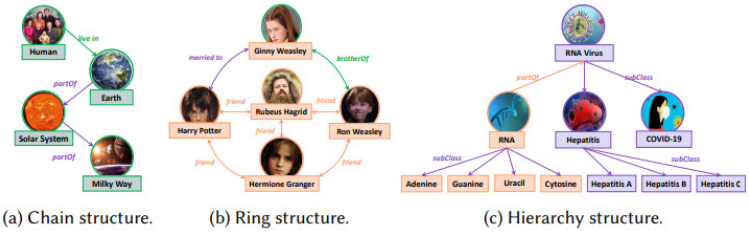
Figure 7.25 – Graph learning tasks (https://arxiv.org/pdf/2404.14809)

Knowledge graph embeddings

KGs are effective in representing knowledge, but they are complex to manipulate at scale. This complicates their use when we are interested in particular tasks such as link prediction and entity classification. Therefore, **knowledge graph embedding (KGE)** was proposed to be able to simplify these tasks. We have already discussed the concept of embedding in the first chapter. An embedding is a projection of data into a low-dimensional and continuous vector space, which is especially useful when our data has a sparse representation (such as text and graphs). For a KG, an embedding is the projection of the graph (nodes, edges, and their feature vectors) in this reduced space. A KGE model then tries to learn a

projection that preserves both structure and information, so that it can then be used for downstream tasks.

Learning this representation is not an easy task, and several types of algorithms have been proposed. For example, some KGEs try to preserve relational patterns between entities. TransE is an approach that embeds KGs in Euclidean space where the relationships between entities are vectors. TransE is based on the idea that two entities connected in the triplet should be as close as possible in space. Furthermore, from a triplet `(h, r, t)`, it tries to learn a space where $h + r \approx t$, thus allowing us to do various mathematical operations. RotatE, another approach, also tries to preserve other relational patterns (symmetry, antisymmetry, inversion, and composition) using a complex vector space. This is quite useful when we want to answer questions that require this notion of symmetry (*marriage* is symmetric) or composition (*my nephew is my brother's son*). Other methods, however, try to preserve structural patterns. In fact, larger KGs contain complex and compound structures that are lost during the embedding process. For example, hierarchical, chain structure, and ring structure are lost during classical embeddings. These structures are important when we want to conduct reasoning or extract subgraphs for some tasks. ATTH (another KG embedding method) uses hyperbolic space to preserve hierarchical structures and logical patterns at the same time. Other methods, however, try to model the uncertainties of entities and relationships, making tasks such as link prediction easier.



(a) Chain structure. (b) Ring structure. (c) Hierarchy structure.

Figure 7.26 – Illustration of three typical structures in KGs (https://arxiv.org/pdf/2211.03536)

KGEs have been used extensively for several tasks such as link prediction. In this case, exploiting the small space is used to try to identify the most likely missing links. Similarly, continuous space allows models to be used to conduct triple classification. A further application is to use learned embeddings to recommender systems.

Graph neural networks

There are several challenges in using graphs natively with ML algorithms. First, classical ML models take data that is in a rectangular or grid-like form, making it non-intuitive how to apply it to graphs. In addition, for a graph, there are several pieces of information that we want to use to solve tasks: nodes, edges, global context, and connectivity. The last one is particularly difficult to represent, and we usually use an adjacency matrix. This representation is sparse, grows largely with the number of nodes in the graph, and thus is space inefficient. Also, since there is no order in the graph, we can get several adjacency matrices that convey the same information but may not be recognized by a model.

A GNN is a deep learning model that natively takes a graph and also exploits its structure during its learning process. There are different types of GNNs (in the *Further reading* section, there are some reviews so you

can go into more detail on this topic) but here we will focus on the main framework: message passing. Most GNNs can be seen as graph convolution networks in which we aggregate for each node the information coming from its neighbors. One of the advantages of a GNN is that at the same time as it is being trained for a task, it learns an embedding for each node. At each step, this node embedding is updated with information from its neighbors.

The message-passing framework is in a sense very similar to a neural network, as we saw earlier. In this case, there are two main steps: gather the embeddings of the various neighboring nodes and then follow by an aggregation function (which can be different depending on various architectures) and a nonlinearity. Then, at each step, we conduct an update of the embedding of each node, learning a new representation of the graph. A classical GNN can be composed of a series of GNN blocks and then a final layer that exploits the learned representation to accomplish a task. This can be written in a formula like this:

$$h_v^{(l+1)} = W^{(l+1)} \cdot \sum_{w \in N(v) \cup \{v\}} \frac{1}{c_{w,v}} h_w^{(l)}$$

Here, at layer *l+1*, we learn a representation, *h*, based on the previous embedding. *W* is a layer-specific weight matrix, *v* is a node, *w* is the set of neighbors, and *c* is the normalization coefficient.

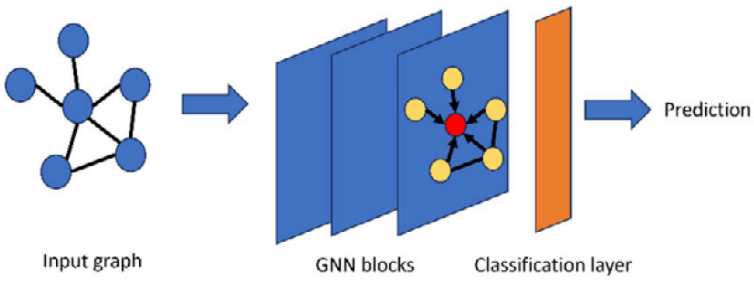


Figure 7.27 – GNNs

In this case, we are assuming that each neighbor's contribution is the same. This may not be the case, so inspired by the attention mechanism of RNNs and transformers, **graph attention networks (GATs)** have been proposed. In this type of GNN, the model learns different levels of importance for each neighbor. Several models of GNN layers exist today, but basically, the principle does not change much.

GNNs have been successfully used for several graph tasks but still have some limitations such as difficult scalability, problems with batching, and so on. They have also been applied to KGs, but they increase complexity.

LLMs reasoning on knowledge graphs

LLMs have the advantage that they are not trained for a specific task but acquire a broad spectrum of skills during training. In addition, LLMs have reasoning skills that can be improved with specific approaches. Therefore, several researchers have suggested conducting graph reasoning with LLMs. The main method of approaching an LLM is to use a prompt as input. There are three approaches:

- **Manual prompt**: The simplest approach is to provide a prompt to an LLM in which they are asked to solve a task on a graph. In the prompt, the graph is entered and additional information can be added (e.g., if

we want the LLM to conduct a **depth-first search** (**DFS**) algorithm to solve the task, we provide a succinct explanation of how this algorithm works). A limitation to these prompts is that it is not possible to insert wide graphs within a prompt (limitation due to the context length of the LLM).

- **Self-prompting**: The LLM conducts a continuous update of the prompt to make it easier for the LLM to solve tasks. In other words, given an original prompt, the LLM conducts prompt updates to better define tasks and how to resolve them. Then, based on the output of the LLM, a new prompt is generated and fed back to the LLM. This process can be conducted multiple times to refine the output.
- **API call prompts**: This type of prompt is inspired by agents, in which the LLM is provided with a set of APIs that it can invoke to conduct reasoning about graphs or other external tools.

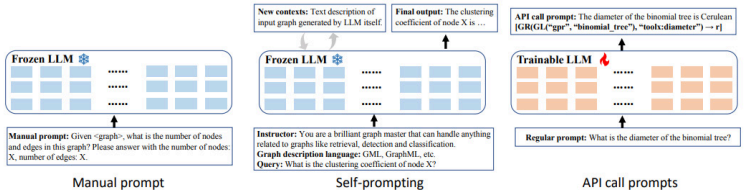


Figure 7.28 – Prompting methods for the LLM applied to graph tasks (**https://arxiv.org/pdf/2404.14809**)

The alternative to finding complex prompting strategies is the **supervised fine-tuning** (**SFT**) method. In this case, a dataset with graph tasks and their solutions is used to train the model to improve its reasoning skills.

Another interesting aspect of LLMs is that they can also be used in combination with other models. This allows their qualities to be exploited with models that are specialized and better suited for certain tasks. For example, we can combine LLMs with GNNs, in which case LLMs can function as enhancers of GNNs. The GNN handles graph structure much better than the LLM, but the latter handles textual attributes much better. We can then capitalize on the strengths of the two models to have a much stronger synergistic model. LLMs possess greater semantic and syntactic capacity than other models, and this allows them to create powerful textual embeddings. An LLM can then generate numerical embeddings that are then used by the GNN as node features. For example, we have a citation network between scientific articles (our graph where each node is an article) and we want to classify articles into various topics. We can take the abstract for each article and use an LLM to create an embedding of the abstract. These number vectors will be the node features for the articles. At this point, we can train our GNN with better results than without features. Alternatively, if the node features are textual, we can use an LLM to generate labels. For example, for our article network, we have the article titles associated with each node, and we use the LLM in a zero-shot setting to generate a set of labels. This is useful because manual annotation is expensive, so we can get labels much more quickly. When we have obtained the labels, we can train a GNN on the graph. Alternatively, we can also think of conducting fine-tuning of the LLM and GNN at the same time for tasks.

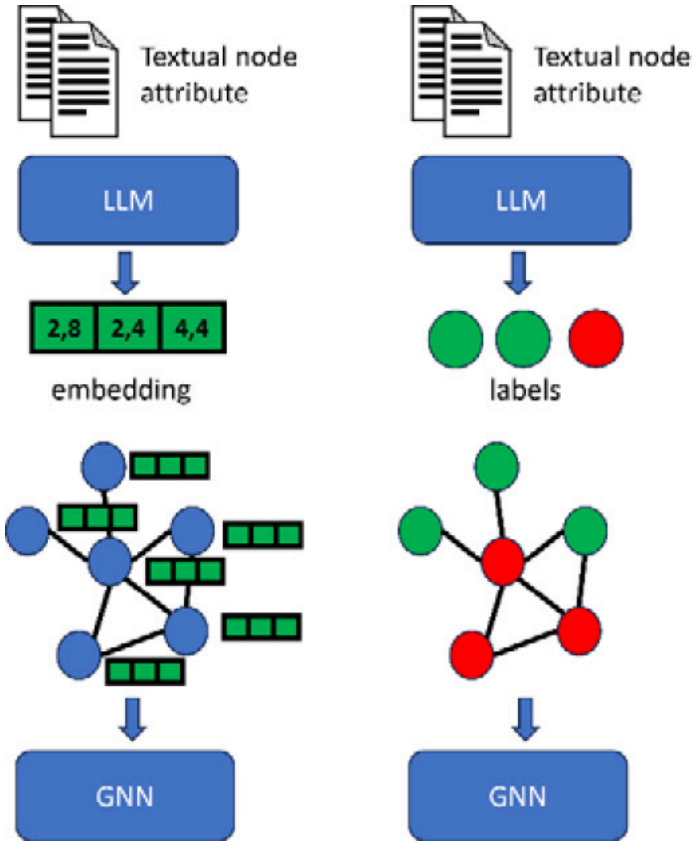


Figure 7.29 – LLM and GNN synergy

Another interesting approach is graph-formed reasoning. Several of the prompting techniques that have been used for reasoning do not take into account that human thinking is not linear, so according to some, this method of reasoning can be approximated using graphs. There are two types of approaches that take advantage of this idea:

- **Think on the graph**: In this approach, the LLM reasons in the form of a graph where each node represents an intermediate of thought (i.e., an intermediate step of reasoning or conclusion during the reasoning necessary to arrive at a solution) and the edges represent the relationship between these thoughts and the direction in which the reasoning is headed. An example of this approach is a **graph of thoughts** (**GoT**), where the reasoning intermediates are presented as a graph. This approach is expensive (multiple calls in inference) and typically used for problems that require reasoning, such as math problems. For example, to solve *A train travels 60 miles in 1.5 hours. What is its average speed?*, the model might create nodes such as **`Distance = 60 km`**, **`Time = 1.5 hours`**, **`Use speed = distance ÷ time`**, and **`Speed = 40 km/h`**, with edges showing how each thought leads to the next. This graph structure enables the model to reason step by step, explore alternatives, or verify calculations.
- **Verify on the graph**: In this approach, we use a graph to verify the correctness and consistency of the reasoning. For example, if different paths should lead to a logical conclusion, they should be the same or similar. So, if there is a contradiction, it means the reasoning is wrong. Generally, one generates several reasonings for a question, structures them as a graph, and analyzes them to improve the final answer. This approach requires a verifier who analyzes this graph, usually another LLM.

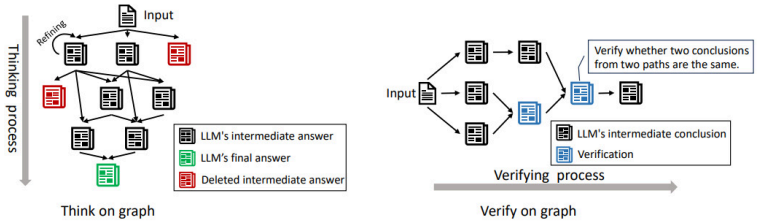


Figure 7.30 – Think on graphs and verify on graphs (https://arxiv.org/pdf/2404.14809)

In this section, we discussed the intricate relationship between graphs and LLMs and how they can enable us to solve some tasks that were previously conducted with graph ML algorithms. In the next section, we will discuss exciting perspectives in the field of some questions that remain open.

Ongoing challenges in knowledge graphs and GraphRAG

KGs are a powerful medium for storing and organizing information, but there are still limitations and open questions. Especially for large KGs, scalability is important; a balance must be struck between expressiveness and computational efficiency. Plus building a KG requires a lot of computational effort (using an LLM to extract triplets from a large corpus of text can be expensive and require adequate infrastructure). In addition, once the KG is built, it must be evaluated and cleaned, which also requires some effort (manual or computational). Moreover, growth in the KG also means growth in the infrastructural cost to enable access or use. Querying large KGs requires having optimized algorithms to avoid the risk of increasingly large latency times. Industrial KGs can contain billions of entities and relationships, representing an intricate and complex scale. Many of the algorithms are designed for small-scale KGs (up to thousands of entities) so retrieval in large-scale KGs still remains challenging.

In addition, KGs are notoriously incomplete, which means that one must have pipelines in order to complete the KG. This means having both pipelines to add additional sources and pipelines to conduct the update of data sources. Most databases are static, so creating dynamic systems is challenging. This is critical to make the best use of KGs for domains such as finance, where we want to account for rapid market changes. On a side note, KGs can be multimodal, but integrating these modes is not easy at all. While adding modalities significantly improves the reasoning process, the understanding of the nuances of stored knowledge, and the richness of the KG, it significantly increases management complexity (more storage required, more sophisticated pipelines, more complex knowledge harmonization, and so on).

GraphRAG is a relatively new technology and still not fully optimized. For one thing, information retrieval could be improved, especially the transition between the user's text query and retrieval on the KG. The more the KG grows, the more there is a risk of finding redundant information that harms the generation process. After retrieval, we could then end up with a long context that is provided to the LLM for generation. To reduce noise and reduce computation, we can compress the context, but this carries the risk of information loss. At present, lossless compression is an active

field of research, while current methods allow, at most, a trade-off between compression and information preservation. There is also currently a lack of benchmark standards to evaluate new GraphRAG approaches; this does not allow for an easy comparison of either current or future methods. GraphRAG allows for considering inter-entity relationships and structural knowledge information, reducing redundant text information, and being able to find global information again. At the same time, though, the nuances of the text are lost, and GraphRAG underperforms in abstractive question-answering tasks or when there is no explicit entity mentioned in the question. So, the union of vector and GraphRAG (or HybridRAG) is an exciting prospect for the future. It remains interesting to understand how these two technologies will be integrated in an optimal way.

An important note is that LLMs are not specifically trained for graph tasks. As we mentioned in ***Chapter 3***, LLMs are trained to predict the next word in a sequence. By optimizing this simple goal, they acquire most of their skills. Obviously, it is difficult to assimilate a spatial understanding simply from text. This means that LLMs generally struggle with structural data. This is highlighted with tabular data, where LLMs have problems with understanding tables and relationships. The first problem is that LLMs struggle with numerical representation since the tokenization step makes it difficult for an LLM to understand the whole number (lack of consistent decimal representation, and problems with numerical operation). This then impacts the execution of graph tasks where this numerical understanding is necessary. Specific studies on graph understanding show that LLMs have a basic understanding of graph structure. LLMs understand these graphs in linear form and better understand the labels associated with the nodes more than the topological structure of the graph. According to these studies, LLMs have a basic understanding, which is strongly impacted by prompt design, prompt techniques, semantic information provided, and the presence of examples. Next-generation, multi-parameter LLMs succeed in solving simple tasks on small graphs, but their performance decays rapidly as both graph and task complexity increase. There are two reasons for this lack of understanding of structural data. The first is that in the large text corpora used for training LLMs, there is not much graph-derived data. So, LLMs can only learn basic spatial relationships because these are described in the texts. Therefore, SFT on graph datasets allows for results that are better than much larger models.

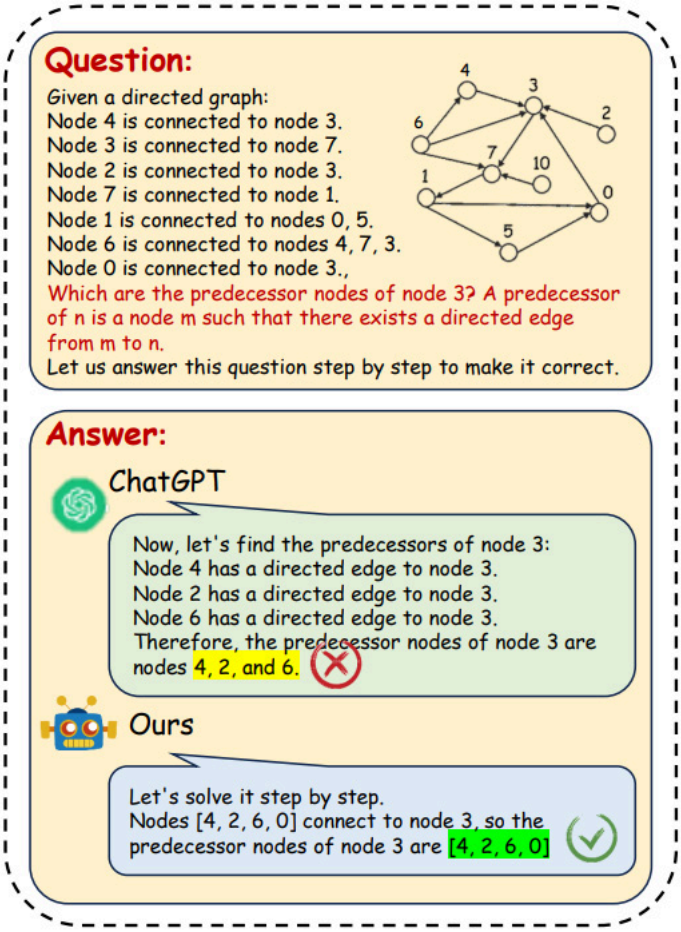


Figure 7.31 – SFT on graph data allows better performance for small LLMs than larger LLMs (https://arxiv.org/pdf/2403.04483)

The second reason, on the other hand, stems from why humans understand spatial structures well. Humans learn spatial relationships from their experiences in the outside world. The brain creates mental maps that allow us to orient ourselves in space. These maps also enable us to better understand abstract spatial concepts such as graphs. LLMs do not have a mental map nor can they have the experience of the outside world, thus making them disadvantaged in understanding abstract spatial concepts.

Summary

In *Chapters 5* and *6*, the main question was how to find information and how to use this information to generate an answer to users' questions. Finding information dynamically allows us to reduce the hallucinations of our model and keep its knowledge up to date.

In this chapter, we started with a text corpus and created a system to find the most relevant information for generating an answer (naïve RAG). Next, we created a more sophisticated system to try to extract only the relevant information and avoid redundant information or noise. For some researchers, by its nature, text contains relevant information intermixed with background noise. What matters are the entities present and their relationships. From this reductionist approach comes the idea of representing essential knowledge in a knowledge graph. The graph allows us to use algorithms to search for information or explore possible connections. For a long time, graph reasoning and LLMs have run on parallel tracks, but recently, their stories have begun to intertwine. We have

seen how this interaction between LLM and KG can be conducted in various ways. For example, an LLM can be used to extract relationships and entities for our graph construction, or an LLM can be used to conduct reasoning about the KG. Similarly, we can use the KG to find knowledge and enrich the context of the LLM, thus enabling it to effectively answer a user question.

Right now, there is a sort of a Manichean definition: either the vector RAG or the GraphRAG. Both have merits and demerits, and the research points toward a unification of these two worlds (HybridRAG). In the future, we will find more sophisticated ways of uniting KGs and vectors. Also, the understanding of the graph structure on one side of an LLM is still immature. With the growth of training datasets, the new generation LLMs are exposed to more examples of graphs. However, understanding spatial relationships in an abstract concept such as a graph also means understanding them in problems with greater real-world relevance. Therefore, this is an active field of research, especially for robots that must interact in space and use AI.

Moving into space is one of the next frontiers of AI. Interaction in space presents peculiar challenges, such as balancing exploration and exploitation. In the next chapter, we will discuss this concept more abstractly. We will focus on reinforcement learning and agent behavior in the relationship to space. Whether chess, a video game, or a real-world environment, an agent must learn how to interact with space to achieve a goal. In the next chapter, we will look at how to enable an agent to explore the world without losing sight of the aim.

Further reading

- Ghafarollahi, *SciAgents: Automating Scientific Discovery through Multi-agent Intelligent Graph Reasoning*, 2024, **https://arxiv.org/pdf/2409.05556v1**
- Raieli, *A Brave New World for Scientific Discovery: Are AI Research Ideas Better?*, 2024, **https://levelup.gitconnected.com/a-brave-new-world-for-scientific-discovery-are-ai-research-ideas-better-5692c5aa8182**
- Raieli, *How the LLM Got Lost in the Network and Discovered Graph Reasoning*, 2024, **https://towardsdatascience.com/how-the-llm-got-lost-in-the-network-and-discovered-graph-reasoning-e2736bd04efa**
- Wu, *Medical Graph RAG: Towards Safe Medical Large Language Model via Graph Retrieval-Augmented Generation*, 2024, **https://arxiv.org/abs/2408.04187**
- Raieli, *The Convergence of Graph and Vector RAGs: A New Era in Information Retrieval*, 2024, **https://medium.com/gitconnected/the-convergence-of-graph-and-vector-rags-a-new-era-in-information-retrieval-b5773a723615**
- Sarmah, *HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction*, 2024, **https://arxiv.org/pdf/2408.04948**
- Liang, *Survey of Graph Neural Networks and Applications*, 2022, **https://onlinelibrary.wiley.com/doi/10.1155/2022/9261537**
- Arora, *A Survey on Graph Neural Networks for Knowledge Graph Completion*, 2020, **https://arxiv.org/pdf/2007.12374**

- Huang, *Can LLMs Effectively Leverage Graph Structural Information through Prompts, and Why?*, 2023, **https://arxiv.org/abs/2309.16595**
- Liu, *Evaluating Large Language Models on Graphs: Performance Insights and Comparative Analysis*, 2023, **https://arxiv.org/abs/2308.11224**

Table of contents collapsed